



深圳職業技術大學

SHENZHEN POLYTECHNIC UNIVERSITY

数据结构课程

堆栈应用括号匹配实验项目

实验实训报告

一、实验实训项目名称

堆栈应用括号匹配实验

二、项目内容与要求

实验目的

- 掌握堆栈的基本原理
- 掌握堆栈的存储结构
- 掌握堆栈的进栈、出栈、判断栈空的实现方法
- 掌握应用堆栈实现括号匹配的原理和实现方法

实验内容

1、问题描述

一个算术表达式中包括圆括号、方括号和花括号三种形式的括号

编程实现判别表达式中括号是否正确匹配的算法

2、算法

顺序扫描算术表达式

若算术表达式扫描完成，此时如果栈空，则正确返回（0）；如果栈未空，说明左括号多于右括号，返回（-3）

从算术表达式中取出一个字符，如果是左括号（‘（’或‘[’或‘{’），则让该括号进栈（PUSH）

如果是右括号：

- （1） 如果栈为空，则说明右括号多于左括号，返回（-2）
- （2） 如果栈不为空，则从栈顶弹出（POP）一个括号：若括号匹配，则转 1 继续进行判断；
否则，说明左右括号配对次序不正确，返回（-1）

3、输入

第一行：样本个数，假设为 n

第二到 n+1 行，每一行是一个样本（算术表达式串），共 n 个测试样本

4、输入样本

4

{[(1+2)*3]-1}

{[(1+2)*3]-1}

$(1+2)*3)-1\}$

$\{[(1+2)*3-1]$

5、输出

共有 n 行，每一行是一个测试结果，有四种结果：

0：左右括号匹配正确 $\{[(1+2)*3]-1\}$

-1：左右括号配对次序不正确 $\{[(1+2]*3)-1\}$

-2：右括号多于左括号 $(1+2)*3)-1\}$

-3：左括号多于右括号 $\{[(1+2)*3-1]$

6、输出样本

0

-1

-2

-3

三、报告内容

1、方法、步骤:

0、头文件说明

```
#include <iostream>
```

```
#include <string.h>
```

```
using namespace std;
```

```
#define ERROR -1
```

```
#define CORRECT 1
```

```
#define MAXSTACKSIZE 100
```

```
// 1、堆栈的定义
typedef struct SqStack {
    char base[MAXSTACKSIZE]; // 栈
    char *top;                // 栈
} SqStack;

SqStack MBStack;
```

```
// 2、初始化堆栈
int InitStack(SqStack &S)
{
    if (S.base == NULL) return(ERROR);
    S.top = S.base;        // 初始化堆
    return(CORRECT);
}
```

```
// 3、进栈
int Push(SqStack &S, char e) // 向栈中放入数据[进栈]
{
    if ((S.top - S.base) >= MAXSTACKSIZE) return(ERROR);
    *S.top++ = e;
    return(CORRECT);
}
```

```
// 4、出栈
int Pop(SqStack &S, char &e) // 从栈中取
{
    if (S.top <= S.base) return(ERROR);
    e = *--S.top;
    return(CORRECT);
}
```

// 5、判断栈空

```
int StackEmpty(SqStack &S)
```

```
{
```

```
    if (S.top <= S.base) return(ERROR);
```

```
    return(CORRECT);
```

```
}
```

// 6、括号匹配

```
int MatchBracket(SqStack &S, char *BracketString) // 判断括号是否
```

```
{
```

```
    int i;
```

```
    char C, sC;
```

```
    InitStack(S);
```

```
    // 清空堆栈
```

```
    for (i=0; i<strlen(BracketString); i++) {
```

```
        C = BracketString[i];
```

```
        if ((C == '(') || (C == '[') || (C == '{')) Push(S, C);
```

```
        if ((C == ')') || (C == ']') || (C == '}')) {
```

```
            if (StackEmpty(S) == ERROR) return(-2); // 右括号多于左括号
```

```
            Pop(S, sC);
```

```
            if ((C == ')') && (sC != '(')) return(-1); // 左右括号不匹配
```

```
            if ((C == ']') && (sC != '[')) return(-1);
```

```
            if ((C == '}') && (sC != '{')) return(-1);
```

```
        }
```

```
    }
```

```
    if (StackEmpty(S) != ERROR) return(-3); // 左括号多于右括号
```

```
    return(0); // 左右括号匹配正确
```

```
}
```

2、实验步骤 (源代码和程序运行截图)

```
// 7、主程序
int main()
{
    int i, SampleNum;
    char BracketString[MAXSTACKSIZE];

    cin >> SampleNum;
    for (i=0; i<SampleNum; i++) {
        cin >> BracketString;
        cout << MatchBracket(MBStack, BracketString) << endl;
    }
    return 0;
}
```

```
PS D:\study\DataStructure> & 'c:\Use
.4-win32-x64\debugAdapters\bin\Window
yxdtdbvq.wla' '--stdout=Microsoft-MIE
e-Error-2ikajo0c.030' '--pid=Microsof
ngw64\bin\gdb.exe' '--interpreter=mi'
4
{[(1+2)*3]-1}
0
{[(1+2]*3)-1}
-1
(1+2)*3)-1}
-2
{[(1+2)*3-1]
-3
```

3、实验结论及思考：

利用栈的后进先出特性，可高效实现括号匹配的顺序验证

